Analysis of BLE Flows and Estimation of the Number of Devices Using RSSI and Temporal Characteristics

Lorenzo Ferrari

October 2026

Contents

1 Introduction **3**

2 Wireshark Packet Analysis **4**

2.1 Packet Extraction and Validation 4

2.2 Advertising Packet Analysis 4

2.3 Connection Identification Using CONNECT_IND 4

2.4 Connection Confirmation 5

2.5 Inference of Unobserved Connections 5

2.6 Communication Flow Analysis 6

3 RSSI and Temporal Analysis for Device Estimation **8**

3.1 Advertising Timing Analysis 9

3.2 MAC Address Candidate Ranking RSSI/TIME 10

3.3 MAC Change Detection and Device Estimation 11

4 Parameter Sensitivity Analysis **12**

4.1 RSSI Tolerance: 5 dB vs 10 dB 12

4.2 Minimum Packets: 10 vs 30 13

4.3 Maximum MAC Gap: 0.5 s vs 2 s 14

5 Conclusion **15**

1 Introduction

In this project we analyze Bluetooth Low Energy (BLE), a communication protocol commonly used in the Internet of Things (IoT).

We developed a Python script with two main objectives:

- Estimate how many devices are communicating.
- Determine how many communication flows are established between devices.

We used BLE communication datasets available on the internet for the analysis. One of the main problems is that BLE devices may change their MAC addresses, so counting the addresses alone is not enough to identify the physical devices.

For this reason, the project uses three main approaches:

- Analyze Wireshark packets to find the devices and the connections that can be observed.
- Use the Received Signal Strength Indicator (RSSI) to help estimate how many devices are communicating, even when MAC addresses change.
- Analyze the timing of the communications to get more information about the flows.

2 Wireshark Packet Analysis

In this part we analyze Wireshark packets to count the connection requests and check which connections can be confirmed from the captured traffic.

2.1 Packet Extraction and Validation

First, the Python script opens the PCAP file and goes through the packets one by one. Packets without the BLE Link Layer (`BTLE`) are ignored. The script also skips packets when the CRC has been checked and is reported as invalid.

2.2 Advertising Packet Analysis

The script looks for advertising PDUs such as `ADV_IND`, `ADV_NONCONN_IND`, `ADV_SCAN_IND`, and `ADV_EXT_IND`.

PDU means Protocol Data Unit. It is the data exchanged between devices at the Link Layer and contains a header and a payload.

For these advertising packets, we extract the advertiser's MAC address, the timestamp and the RSSI, when they are available. We also count how many advertising packets were captured and how many different MAC addresses appear.

2.3 Connection Identification Using CONNECT_IND

The next step is to find `CONNECT_IND` packets, identified by PDU type `0x05`. From each packet we extract the initiator's MAC address, the advertiser's MAC address and the Access Address.

The Access Address is important because it lets us associate the following Data PDUs with the same connection.

2.4 Connection Confirmation

After finding the Data PDUs for an Access Address, the script checks the `NESN` (Next Expected Sequence Number) field. This field is used for acknowledgments during BLE communication. A change in its value is taken as evidence that packets are being exchanged.

The script also looks for `LL_FEATURE_RSP`, which is sent when a device responds with the Link Layer features it supports.

If either of these is observed, the connection is marked as `confirmed`. Otherwise, it remains `unverified`.

We also keep track of connection interval updates using `LL_CONNECTION_UPDATE_IND` and of connection termination messages using `LL_TERMINATE_IND`.

2.5 Inference of Unobserved Connections

Sometimes a connection starts before the capture begins, or the sniffer misses its `CONNECT_IND`. In these cases we may still see Data PDUs, but not the packet that started the connection.

To handle this, the script checks Data PDUs whose Access Address does not belong to a previously observed `CONNECT_IND`.

Packets with the same Access Address are grouped into possible connection events using their timestamps and radio channels. We start a new event if the gap between two packets is greater than 5 ms or if the radio channel changes.

Next, we look for a repeated interval between the events. Since BLE connection intervals are multiples of 1.25 ms, the script searches for intervals that follow this pattern. There must be at least five observed events, with the same compatible interval appearing at least four times.

This is not enough on its own. The script also requires at least one of these conditions:

- At least two pairs of packets close to each other in time, which may

indicate communication between two devices.

- Both possible values of the `NESN` field have been observed, suggesting acknowledgment activity.

When the conditions are satisfied, the connection is classified as `inferred`.

This does not prove that the connection started before the capture. It is also possible that the sniffer simply missed the `CONNECT_IND` packet.

2.6 Communication Flow Analysis

Once the confirmed and inferred connections have been identified, the script analyzes when they were active and what information was observed for each one.

For confirmed connections, it reports the Access Address, the number of Data PDUs, the connection interval when available, and whether a termination packet was captured.

For inferred connections, it reports the Access Address, the number of packets, the recurring interval that was found, and how many radio channels were observed.

We also compare the start and end times of the connections to find the maximum number of communication flows that were active at the same time. For confirmed connections, we use the addresses in `CONNECT_IND` to count the known endpoints.

At the end, the script reports three values:

- **Connection requests:** the number of observed `CONNECT_IND` packets.
- **Confirmed connections:** requests for which we found additional evidence of communication.
- **Inferred connections:** possible connections found without observing their `CONNECT_IND` packet.

This analysis tells us about the observed communication flows, but it does not give the exact number of physical devices. Different connections can involve the same devices, and a device can change its MAC address.

For this reason, the next part uses RSSI and timing information to improve the estimate of how many devices are communicating.

3 RSSI and Temporal Analysis for Device Estimation

In this part we try to estimate the number of physical BLE devices, even when a device changes its MAC address. BLE devices can use randomized addresses to make tracking more difficult.

For example, consider these captured packets:

Time	**MAC**	**RSSI**
1.0 s	AA:AA	-65 dBm
2.0 s	AA:AA	-66 dBm
3.0 s	AA:AA	-64 dBm
3.2 s	BB:BB	-65 dBm
4.0 s	BB:BB	-66 dBm

If we only count the different addresses, we find two MACs. However, it is also possible that the same device changed its address from `AA:AA` to `BB:BB` after three seconds.

Instead of looking only at the address, our script compares:

- RSSI (Received Signal Strength Indicator).
- RSSI distribution and variability.
- Advertising timing characteristics.
- How close the two MAC addresses are in time.

These comparisons do not prove that two addresses belong to the same device. They are used to find possible associations based on the received signal and the timing of the packets.

RSSI is the signal strength measured by the receiver, usually expressed in dBm. When a device stays in approximately the same position, we can

expect its RSSI values to remain relatively similar even if the MAC address changes.

In the `analyze()` function, the script reads RSSI from the `BTLE_RF` or `NORDIC_BLE` fields. It checks that a MAC address is available and that the RSSI is between -127 and 20. Each valid sample is saved as a tuple containing `(timestamp, MAC, RSSI)`.

The samples are then grouped by MAC address and sorted by time. For each MAC, we take two windows of two seconds: one at the beginning and one at the end of the observed activity.

For both windows we calculate:

- Mean.
- Median.
- Variance.
- Standard deviation.

Mean and median describe the typical RSSI value, while variance and standard deviation show how much it changes. We use these statistics to compare the last RSSI profile of an old MAC with the first profile of a new one.

3.1 Advertising Timing Analysis

A BLE device can send several advertising packets on different channels during the same advertising event. To avoid treating all these packets as separate events, the script groups packets that are close in time, using a 5 ms threshold.

After that, it calculates the time between consecutive advertising events by subtracting their timestamps. We calculate the mean, median and standard deviation of these intervals and compare them for different MAC addresses.

For example, if one MAC sends advertising packets about every 100 ms and a second MAC follows a similar interval, this can be another indication that both addresses belong to the same device.

3.2 MAC Address Candidate Ranking RSSI/TIME

First, the script considers each MAC as a possible new address and searches for an older MAC to compare it with. Before comparing a pair, we check some conditions:

- Both the beginning and the end of each MAC's observations must contain at least 10 RSSI samples.
- The observation periods of the two MAC addresses must not overlap.
- The new MAC must appear no more than 0.5 seconds after the old one disappears.
- There must be at least two seconds of capture coverage after the first observation of the new MAC.

For the valid candidates, the script compares the RSSI profiles. It calculates the absolute difference between the mean, median and standard deviation of the two profiles. Each difference is divided by `RSSI_TOLERANCE`, and then the average is used as the RSSI distance. A smaller value means that the two profiles are more similar.

We do a similar comparison for the advertising intervals. The script calculates the differences between their mean, median and standard deviation. Each difference is normalized using the larger of the two values, and their average gives the timing distance. This tells us how similar the advertising periodicities are.

Finally, the score is based on three values:

- **RSSI distance:** how similar the RSSI distributions are.

- **Timing distance:** how similar the advertising intervals are.
- **Time gap:** the time between the disappearance of the old MAC and the appearance of the new one, divided by the maximum allowed gap.

The script calculates the score as the average of these values. If the timing distance is not available, it is not included. We then order the candidates by score, starting from the lowest one. A lower score means a more similar pair, but we do not accept the association just because it is ranked first.

3.3 MAC Change Detection and Device Estimation

After ranking the candidates, we check whether the best one can be accepted as a possible MAC change. The script uses these conditions:

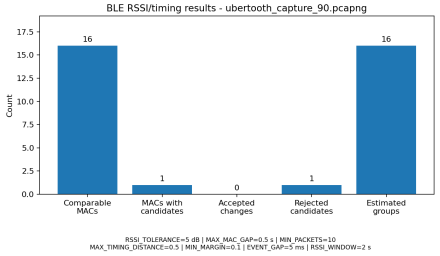
- **RSSI threshold:** the difference between the RSSI medians must be at most 5 dB.
- **Timing threshold:** the normalized timing distance must be at most 0.5, when it is available.
- **Score margin:** if there is another candidate, the best score must be at least 0.1 lower than the second-best one.
- **Mutual preference:** the new MAC must select the old MAC as its best predecessor, and the old MAC must select the new one as its best successor. When alternatives exist, the score margin must also be large enough in the reverse direction.

If all these checks pass, we accept the pair as a possible MAC address change. It is still only an estimate: two different devices can have similar RSSI values and advertising timing, so we cannot be sure that the two addresses belong to the same physical device.

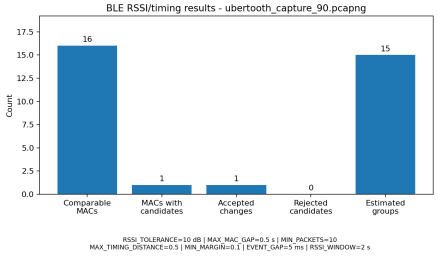
4 Parameter Sensitivity Analysis

We also analyzed how changing some parameters affects the results of our algorithm. For each test, we changed only one parameter while keeping the others fixed. We used the same capture for both settings of each parameter.

4.1 RSSI Tolerance: 5 dB vs 10 dB



RSSI tolerance: 5 dB

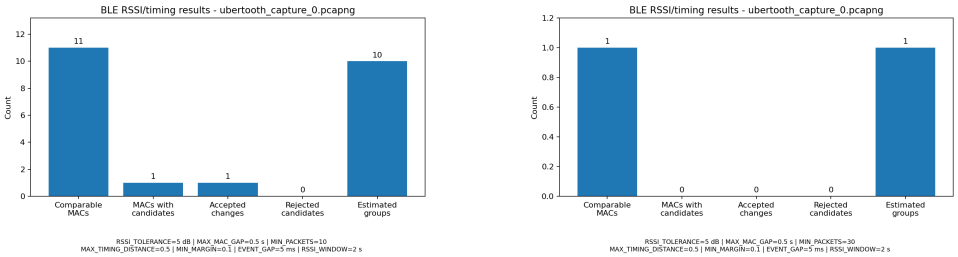


RSSI tolerance: 10 dB

Figure 1: Results obtained by changing the RSSI tolerance from 5 to 10 dB.

In the first test, we increased `RSSI_TOLERANCE` from 5 to 10 dB. With the original value, no MAC changes were accepted. After increasing the tolerance, one possible MAC change was accepted, reducing the estimated groups from 16 to 15. This happened because the new threshold allows a larger difference between the RSSI values of two MAC addresses. It may help recover some changes, but could also accept incorrect associations.

4.2 Minimum Packets: 10 vs 30

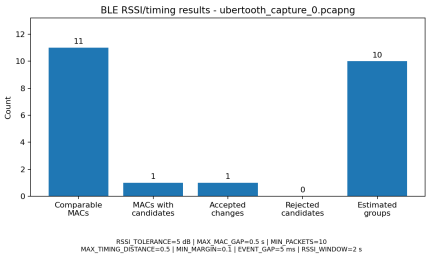


MIN_PACKETS = 10 MIN_PACKETS = 30

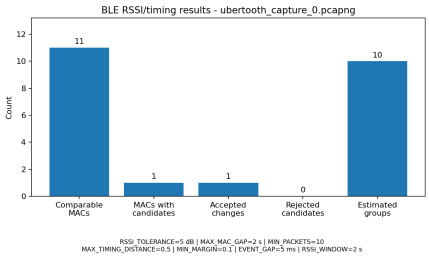
Figure 2: Comparison of minimum required RSSI samples.

In the second test, we changed `MIN_PACKETS` from 10 to 30 samples per window. With 10 samples, the script found 11 comparable MAC profiles and accepted one possible MAC change, giving 10 estimated groups. With 30 samples, only one profile met the requirement and no MAC changes were accepted, giving one estimated group. This happens because more MAC addresses are excluded when we require more samples. It does not mean that the number of physical devices decreased: the script simply has fewer profiles it can compare.

4.3 Maximum MAC Gap: 0.5 s vs 2 s



Maximum gap: 0.5 s



Maximum gap: 2 s

Figure 3: Results obtained by increasing the maximum gap between MAC addresses from 0.5 to 2 seconds.

Finally, we increased `MAX_MAC_GAP` from 0.5 to 2 seconds. The number of accepted MAC changes and estimated groups remained the same. The existing association already satisfied the shorter limit, and the longer gap did not result in any additional accepted associations. However, the similarity scores changed because this parameter is also used to normalize the time gap between two MAC addresses.

5 Conclusion

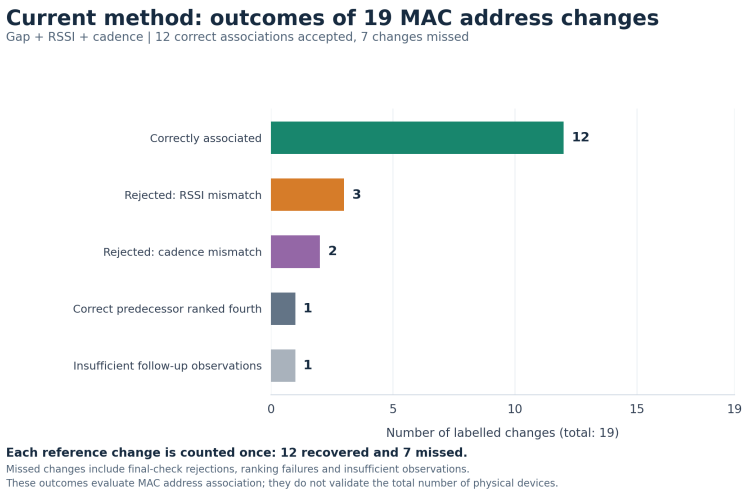


Figure 4: Results of MAC address change detection.

In the first figure we show the results for 19 known MAC address changes. The script correctly associated 12 of them, which is about 63.2%. The other 7 changes were missed.

Of these 7 cases, 3 were rejected because the RSSI values were too different, and 2 because the advertising intervals were too different. In one case the correct previous MAC was only fourth in the ranking. In another case there were not enough observations to make the association.

These results show that RSSI and timing can help recognize possible MAC changes. At the same time, changes in radio conditions and the thresholds used by the script can cause real changes to be missed.

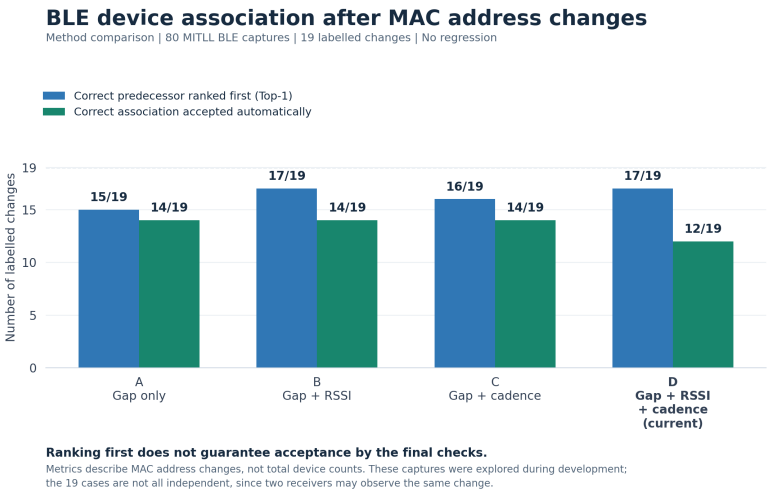


Figure 5: Comparison of different MAC association methods.

The second figure compares four versions of the method: gap only, gap with RSSI, gap with advertising cadence, and the version that uses all three characteristics.

With the complete method, the correct previous MAC was ranked first in 17 out of 19 cases. However, only 12 associations passed the final checks, while each of the simpler versions accepted 14 correct associations.

This means that using more information can help the ranking, but the extra checks can also reject real MAC address changes.

In conclusion, the script gives us information about BLE communication flows and uses RSSI and timing to find possible changes of MAC address. These characteristics can help with estimating device activity, but they are not enough to know the exact number of physical devices with certainty.